

Lista integrada de programação 2 - Reforço de programação

1) Manipulação E Formatação De Strings (Cadeias De Caracteres)

As strings em Python são dados compostos imutáveis, uma variável que contém outros dados “menores” dentro dela. Uma string comporta outras strings, ou: Uma palavra (String como um todo) comporta letras (Strings individuais).

Exercício S-1: Limpeza e Formatação de Dados de Movimento

Problema: Um sensor de velocidade na pista do IFC registrou dados brutos com espaçamentos excessivos digitados por engano. Escreva uma função chamada 'limpa_dados_velocidade' que receba uma string de entrada contendo espaços em branco extras no início e no fim (por exemplo: ' 85.45 ') e:

1. Remova todos os espaços em branco excedentes.
2. Converta a string limpa para um número de ponto flutuante (float).
3. Retorne uma string formatada contendo exatamente duas casas decimais seguida da unidade de medida 'm/s'. Exemplo: Entrada: ' 12.3456 ' => Saída: 'Velocidade Limpa: 12.35 m/s'.

Exercício S-2 : Análise Biológica de Cadeias de DNA

Problema: Cadeias de DNA são strings contendo as letras 'A', 'T', 'C' e 'G'. Crie uma função chamada 'analisa_dna' que receba uma string representando uma cadeia de DNA em formato qualquer (exemplo: 'atgCtgCtaaAtcg ') e realize as seguintes etapas:

1. Remova espaços em branco extras e converta toda a cadeia para letras maiúsculas.
2. Calcule e exiba o tamanho total da cadeia de DNA limpa.
3. Use o fatiamento (slicing) de strings para obter e exibir:
 - a) O primeiro códon (os primeiros 3 caracteres);
 - b) O último códon (os últimos 3 caracteres).
4. Conte quantas vezes a base Adenina ('A') aparece na fita e exiba a quantidade.

Exercício S-3: Processamento Geológico subterrâneo (Split/Join)

Problema: Uma sonda subterrânea registrou as camadas do interior da Terra separadas por hashtags: 'Crosta#Astenosfera#Mesosfera#Nucleo Externo#Nucleo Interno'. Escreva um programa que faça o seguinte:

1. Receba essa string original.
2. "Quebre" a string a partir do caractere '#' e transforme em uma lista de strings.
3. Com a lista gerada, use o método '.join()' para reconstruir uma string elegante onde as camadas estão separadas por uma seta e espaços (' -> '). Exemplo de saída: 'Crosta -> Astenosfera -> Mesosfera -> ...'.

2) Listas, Mutabilidade e Referência (Dados Compostos)

Diferente das strings, as listas em Python são estruturas de dados mutáveis, o que significa que podemos modificar, adicionar ou remover elementos diretamente em sua memória. Nesta seção, exploraremos os métodos internos de lista, clonagem de fatias e o comportamento de passagem de listas por referência.

Exercício L-1: Análise Estatística de Notas Escolares

Problema: Crie uma função chamada 'analisar_notas' que receba uma lista contendo as notas de matemática de um estudante (exemplo: [8.5, 6.0, 7.0, 9.5, 5.0]). A sua função deve realizar as seguintes ações sem alterar a ordem da lista original recebida:

1. Crie uma cópia ordenada da lista.
2. Apresente as notas em ordem crescente.
3. Encontre e exiba a menor nota e a maior nota.
4. Calcule e exiba a média final do aluno.
5. Use o operador de pertencimento 'in' para verificar se a nota 7.0 está presente na lista.

3) Estruturas De Repetição (Loops While e For)

Laços de repetição automatizam tarefas idênticas ou similares de forma altamente eficiente. Aqui exploraremos loops iterativos 'while' (finitos e infinitos sob condições de guarda) e laços 'for' que varrem estruturas de coleções indexadas usando ferramentas como 'range()' e 'enumerate()'.

Exercício R-1: Tabela de Diluição e Decaimento de Elementos

Problema: Um elemento químico radioativo perde 50% de sua massa a cada minuto de decaimento. Escreva um programa utilizando um laço 'while' que solicite ao usuário uma massa inicial M (em gramas). O loop deve continuar dividindo a massa por 2 e incrementando o tempo em minutos até que a massa total fique abaixo de 0.05 gramas. O programa deve exibir, no final, a tabela passo a passo e o tempo total em minutos necessário para atingir esse limite crítico de estabilidade.

Exercício R-2: Gerador de Padrões Gráficos com Laços Alinhados

Problema: Laços aninhados (loops dentro de loops) são excelentes para gerar matrizes e padrões textuais geométricos. Escreva um programa em Python que use um laço 'for' para gerar o seguinte padrão de escada numérica na tela, onde o número máximo de linhas é fornecido pelo usuário:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

4) Funções Frutíferas e Recursividade

Funções estruturam e reutilizam blocos lógicos. Nesta seção, consolidamos a diferença entre procedimentos vazios e funções frutíferas que devolvem valores utilizando a palavra reservada 'return'.

Exercício F-1: Simulador de Juros Compostos

Problema: Escreva uma função frutífera chamada 'calcula_juros_compostos' que receba três parâmetros: capital inicial (C), taxa de juros anual (i) e quantidade de anos (t). A função deve calcular o montante acumulado no regime de juros compostos utilizando a fórmula matemática:

$$M = C * (1 + i)^t$$

Importante: A sua função deve ser estritamente frutífera (usar o comando 'return' para enviar o montante calculado de volta) e não deve conter comandos print() dentro dela. No corpo principal do programa, chame a função, receba o resultado e exiba com o formato financeiro do Real ('R\$').

Exercício F-2: O Somatório Recursivo de Dígitos de um Número

Problema: A recursividade consiste em fazer com que uma função chame a si própria para resolver uma instância menor do mesmo problema, terminando sempre em um 'caso base' conhecido. Escreva uma função recursiva chamada 'soma_digitos_recursiva(n)' que calcule e retorne a soma dos dígitos individuais de um número inteiro positivo N.

Exemplo:

Entrada: 1234 -> Saída: 1 + 2 + 3 + 4 = 10

Dicas lógicas para a recursão:

Caso recursivo: a soma dos dígitos é o último dígito (N % 10) somado à chamada recursiva contendo os demais dígitos à esquerda, que são obtidos dividindo o número de forma inteira por 10 (N // 10).